



Learner Guide

For

Google Associate Cloud Engineer Training

Course Code TGS-2023041024

TSC Cloud Computing · ICT-DIT-3020-1.1

Synchronous e-Learning

Conducted by Tertiary Infotech Academy Pte Ltd

UEN 201200696W

Version v8.0 · 12 September 2026

Document Version Control Record

Version Number	Effective Date of Release	Summary of Included Changes	Author
7.0	Legacy release	Original reference deck and course materials	Tertiary Infotech
8.0	12 September 2026	Rebuilt Learner Guide: current ACE domains, technical mechanisms, ten Activities, detailed verification and evidence guidance	Dr Alfred Ang

Table of Contents

How to Use This Guide.....	4
Google Cloud Account and Console Setup.....	6
Topic 1 Google Cloud and Cloud Computing.....	8
Activity 01 Safe Training Environment.....	10
Topic 2 GCP Basics and IAM.....	12
Activity 02 Least-Privilege IAM.....	15
Topic 3 Virtual Networks.....	18
Activity 03 Custom VPC.....	20
Topic 4 Virtual Machines.....	22
Activity 04 Compute Engine Recovery.....	23
Topic 5 Storage and Databases.....	25
Activity 05 Storage and Cloud SQL.....	26
Topic 6 Interconnecting Networks.....	28
Activity 06 Hybrid Connectivity.....	30
Topic 7 Managed Services.....	32
Activity 07 Pub/Sub to Cloud Run.....	34
Topic 8 Kubernetes.....	36
Activity 08 GKE Autopilot.....	38
Topic 9 Resource Management and Monitoring.....	40
Activity 09 Incident Evidence.....	42
Topic 10 Load Balancing and Autoscaling.....	44
Activity 10 MIG Load Test.....	46
Assessment Preparation and References.....	48

How to Use This Guide

This guide provides the detailed procedures, expected evidence, troubleshooting logic and cleanup controls for the ten Activities. The slide deck explains mechanisms and decision rules; this guide is the authoritative click-by-click and command-level learner procedure.

Use only the assigned training project. Confirm account, project, region and cost guardrails before each mutation. Never submit credentials, payment details, private keys or access tokens.

Course Identity and Outcomes

Field	Value
Course	Google Associate Cloud Engineer Training
Code	TGS-2023041024
TSC	Cloud Computing · ICT-DIT-3020-1.1
Duration	4 days · 30 training hours · 2 assessment hours
Delivery	Synchronous e-Learning

- LO1: Set up and test Google Cloud solutions in accordance with security standards.
- LO2: Detect and resolve Google Cloud integration issues using technical solutions and database administration techniques.
- LO3: Conduct implementation tests against defined metrics and escalate unresolved Google Cloud issues with evidence.

Knowledge and Ability Mapping

Code	Statement
K1	Security and quality standards
K2	Diagnostic tools for cloud-related problems
K3	Types of installation tests for cloud solutions
K4	Technical solutions or techniques to resolve cloud integration-related issues
K5	Database administration and maintenance tools and techniques
A1	Set up an environment for implementation of cloud solutions in accordance with user

Code	Statement
	requirements
A2	Run tests for cloud solutions to minimise integration-related issues
A3	Detect issues with cloud solutions to identify solutions
A4	Resolve cloud integration and implementation issues
A5	Conduct implementation tests to measure effectiveness against defined metrics
A6	Escalate unresolved implementation issues with reproducible evidence

Google Cloud Account and Console Setup

The following procedure is part of Activity 1 and must be completed before billable services are used. The screens and terms can change; use the current labels in the Google Cloud console and pause if they differ materially.

Step 1

Open <https://cloud.google.com/> and select Get started for free. Sign in with a Google Account you control. Create one first if required, then enable multi-factor authentication in the Google Account security settings.

Step 2

Review the current trial eligibility, country, payment and automatic-upgrade terms. Do not use a corporate billing account unless your organisation has approved the training spend.

Step 3

Accept the Google Cloud terms. If prompted, create a Cloud Billing account and add the required payment method. Record only the billing account name; never place card details in an activity submission.

Step 4

Open <https://console.cloud.google.com/>. Use the project selector at the top and choose New Project. Name it ace-training-<your initials>. Copy the generated project ID exactly; the display name and project ID are different.

Step 5

From Billing, link only the new training project to the training billing account. Verify the Billing overview names the same project ID.

Step 6

From Billing > Budgets & alerts, create ACE training budget. Set a deliberately small amount agreed by the trainer and add thresholds at 50%, 90% and 100%. Remember that a budget alerts but does not stop spending.

Step 7

Open Cloud Shell from the console header. Wait for the terminal prompt, then run `gcloud auth list`. Confirm the active identity is yours.

Step 8

Set `PROJECT_ID` to the copied project ID, export it for this shell, and run `gcloud config set project $PROJECT_ID`. Run `gcloud config list` and confirm both account and project.

Step 9

Enable the Compute Engine, Service Usage, Cloud Logging and Cloud Monitoring APIs. Wait for the operation to finish and list enabled services.

Step 10

Run the supplied `check_environment.sh`. Save its output as `environment-check.txt`. Capture the project ID, billing enabled state, enabled APIs and timestamp.

Step 11

Negative test: switch the script to a deliberately invalid project ID and record the error. Restore the correct project immediately. Explain how this test prevents accidental work in the wrong project.

Step 12

Open Billing reports and record the current cost baseline. Remove any test resource created accidentally. Keep the project for subsequent activities.

```
gcloud auth list
gcloud config set project "$PROJECT_ID"
gcloud config list
gcloud services list --enabled --project "$PROJECT_ID"
```

Verification: the active identity belongs to you, the active project ID is the assigned training project, billing is linked, budget thresholds exist, and required APIs are enabled. A budget alert does not cap spending.

Topic 1 Google Cloud and Cloud Computing

Mapped criteria A2 · K3. ACE domain: Set up a cloud solution environment.

Projects as operating and trust boundaries

Architecture: Project ID, enabled APIs, billing link, IAM policy.

Runtime trace: create → link billing → enable API → deploy → verify.

```
gcloud projects describe $PROJECT_ID
gcloud services list --enabled --project=$PROJECT_ID
```

Acceptance evidence: Project state is ACTIVE; intended APIs are enabled; resources resolve to the same project ID.

Common failure modes: wrong active project; billing not linked; API disabled.

Regions, zones and resource scope

Architecture: global resources, region, zone, multi-region.

Runtime trace: requirement → scope check → location choice → deploy → failover test.

```
gcloud compute zones list --filter='region:(asia-southeast1) '
gcloud compute regions describe asia-southeast1
```

Acceptance evidence: Location choice matches latency, residency and availability requirements.

Common failure modes: zonal single point; unsupported service location; cross-region egress.

Shared-responsibility control map

Architecture: Google infrastructure, managed service, customer configuration, customer data.

Runtime trace: classify service → assign owner → set control → collect evidence → review.

```
gcloud asset search-all-resources --scope=projects/$PROJECT_NUMBER --
format='table(assetType,location) '
```

Acceptance evidence: Every resource has an accountable owner and a testable configuration control.

Common failure modes: assuming managed means secured; unowned data; no evidence trail.

Console, Cloud Shell and local CLI surfaces

Architecture: Cloud console, Cloud Shell, gcloud CLI, REST API.

Runtime trace: authenticate → select config → issue request → receive operation → inspect result.

```
gcloud auth list
gcloud config configurations list
gcloud config list
```

Acceptance evidence: The intended account and project are active before any mutation.

Common failure modes: wrong account; wrong configuration; expired credentials.

API enablement and service identities

Architecture: Service Usage API, service agent, IAM grant, service endpoint.

Runtime trace: request enable → long-running operation → service agent created → permission checked → API ready.

```
gcloud services enable compute.googleapis.com monitoring.googleapis.com
logging.googleapis.com
```

Acceptance evidence: Enabled service appears in the service list and its agent receives only documented roles.

Common failure modes: SERVICE_DISABLED; permission denied; quota project mismatch.

Billing attachment and budget signals

Architecture: billing account, project, budget, notification channel.

Runtime trace: link → record baseline → set thresholds → emit forecast → act.

```
gcloud billing projects describe $PROJECT_ID
gcloud billing budgets list --billing-account=$BILLING_ACCOUNT
```

Acceptance evidence: Billing is enabled and budget thresholds are visible before billable resources are created.

Common failure modes: budget is not a cap; closed account; wrong currency assumption.

Installation test pyramid for cloud resources

Architecture: configuration test, connectivity test, load test, failover test.

Runtime trace: define expected → deploy → observe → compare → record.

```
gcloud asset search-all-resources --scope=projects/$PROJECT_NUMBER --format=json >
inventory.json
```

Acceptance evidence: Each test has expected output, observed output, timestamp and owner.

Common failure modes: test without baseline; single happy path; no rollback.

ACE task-domain traceability

Architecture: environment 20%, planning 17.5%, deploying 25%, operations 20%, security 17.5%.

Runtime trace: read objective → map service → practice command → diagnose failure → verify evidence.

```
gcloud version
gcloud info --format='value(config.paths.global_config_dir)'
```

Acceptance evidence: Practice covers both console interpretation and CLI evidence across all five exam domains.

Common failure modes: memorising product names; ignoring operations; no hands-on evidence.

Activity 01 Create a Safe Google Cloud Training Environment

Duration 75 minutes. Mapped criteria A1, A2, K1, K3.

Create an account, project, billing guardrails and CLI configuration, then prove the environment is safe to use.

Submit: environment-check.txt, budget-policy.json and a project inventory

Files supplied

File	Use
check_environment.sh	Mock data, script or configuration
budget-policy.json	Mock data, script or configuration

Detailed procedure

Step 1

Open <https://cloud.google.com/> and select Get started for free. Sign in with a Google Account you control. Create one first if required, then enable multi-factor authentication in the Google Account security settings.

Step 2

Review the current trial eligibility, country, payment and automatic-upgrade terms. Do not use a corporate billing account unless your organisation has approved the training spend.

Step 3

Accept the Google Cloud terms. If prompted, create a Cloud Billing account and add the required payment method. Record only the billing account name; never place card details in an activity submission.

Step 4

Open <https://console.cloud.google.com/>. Use the project selector at the top and choose New Project. Name it ace-training-<your initials>. Copy the generated project ID exactly; the display name and project ID are different.

Step 5

From Billing, link only the new training project to the training billing account. Verify the Billing overview names the same project ID.

Step 6

From Billing > Budgets & alerts, create ACE training budget. Set a deliberately small amount agreed by the trainer and add thresholds at 50%, 90% and 100%. Remember that a budget alerts but does not stop spending.

Step 7

Open Cloud Shell from the console header. Wait for the terminal prompt, then run `gcloud auth list`. Confirm the active identity is yours.

Step 8

Set PROJECT_ID to the copied project ID, export it for this shell, and run gcloud config set project \$PROJECT_ID. Run gcloud config list and confirm both account and project.

Step 9

Enable the Compute Engine, Service Usage, Cloud Logging and Cloud Monitoring APIs. Wait for the operation to finish and list enabled services.

Step 10

Run the supplied check_environment.sh. Save its output as environment-check.txt. Capture the project ID, billing enabled state, enabled APIs and timestamp.

Step 11

Negative test: switch the script to a deliberately invalid project ID and record the error. Restore the correct project immediately. Explain how this test prevents accidental work in the wrong project.

Step 12

Open Billing reports and record the current cost baseline. Remove any test resource created accidentally. Keep the project for subsequent activities.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 2 GCP Basics and IAM

Mapped criteria A1 · K1. ACE domain: Configure access and security.

Resource hierarchy and inherited policy

Architecture: organization, folder, project, service resource.

Runtime trace: attach policy → inherit downward → combine bindings → evaluate deny → authorize.

```
gcloud projects get-iam-policy $PROJECT_ID --format=json
```

Acceptance evidence: Effective access includes direct and inherited allow policies, then applies applicable deny constraints.

Common failure modes: granting at too-wide scope; hidden inherited role; project outside organization.

IAM authorization tuple

Architecture: principal, role, resource, allow policy.

Runtime trace: authenticate → resolve permission → collect policies → evaluate → allow or deny.

```
gcloud projects add-iam-policy-binding $PROJECT_ID --member=user:learner@example.com --role=roles/viewer
```

Acceptance evidence: Binding identifies one principal, one role and the narrowest practical resource scope.

Common failure modes: basic Owner role; misspelled principal; binding on wrong project.

Predefined, custom and basic roles

Architecture: permissions, predefined role, custom role, basic role.

Runtime trace: list required actions → map permissions → select role → grant → retest.

```
gcloud iam roles describe roles/storage.objectViewer
gcloud iam roles list --project=$PROJECT_ID
```

Acceptance evidence: Selected role contains required permissions without unrelated administrative power.

Common failure modes: role drift; unsupported custom permission; basic role in production.

Service accounts as workload principals

Architecture: service account, attached workload, access token, target API.

Runtime trace: workload identity → metadata token → IAM check → API call → audit log.

```
gcloud iam service-accounts create app-runtime --display-name='App runtime'
gcloud projects add-iam-policy-binding $PROJECT_ID --member=serviceAccount:app-runtime@$PROJECT_ID.iam.gserviceaccount.com --role=roles/storage.objectViewer
```

Acceptance evidence: Workload uses an attached user-managed service account; no long-lived key file is required.

Common failure modes: default service account; downloaded key; token audience mismatch.

Service account impersonation

Architecture: operator, Token Creator role, short-lived token, target service account.

Runtime trace: authenticate operator → authorize impersonation → mint token → call API → expire.

```
gcloud auth print-access-token --impersonate-service-account=app-  
runtime@$PROJECT_ID.iam.gserviceaccount.com
```

Acceptance evidence: Token is short-lived and the impersonation event is auditable.

Common failure modes: missing Token Creator; self-impersonation confusion; cached wrong identity.

Organization policy and deny guardrails

Architecture: constraint, policy rule, resource hierarchy, enforcement result.

Runtime trace: define constraint → set policy → inherit → evaluate request → block or allow.

```
gcloud resource-manager org-policies describe constraints/compute.vmExternalIpAccess --  
project=$PROJECT_ID
```

Acceptance evidence: Policy state and inheritance explain why a request is allowed or rejected.

Common failure modes: constraint unset; policy attached above project; exception not documented.

IAM Conditions and time-bound access

Architecture: role binding, CEL expression, request context, resource.

Runtime trace: request → evaluate time/resource → grant conditional → audit → expire.

```
gcloud projects get-iam-policy $PROJECT_ID --  
format='table(bindings.role,bindings.members,bindings.condition) '
```

Acceptance evidence: Condition title, expression and scope match the intended temporary access window.

Common failure modes: condition on unsupported role; timezone error; broad inherited grant remains.

Audit evidence for access changes

Architecture: Admin Activity log, principalEmail, methodName, resourceName.

Runtime trace: change policy → write audit log → route → query → review.

```
gcloud logging read 'protoPayload.methodName:SetIamPolicy' --limit=20 --format=json
```

Acceptance evidence: Evidence identifies who changed which policy, when, and on which resource.

Common failure modes: query too broad; wrong project; data-access logs not enabled where needed.

Activity 02 Implement Least-Privilege IAM

Duration 75 minutes. Mapped criteria A1, K1.

Create a workload service account, grant a narrow role and verify both allowed and denied actions.

Submit: iam-policy-before.json, iam-policy-after.json and access-test.txt

Files supplied

File	Use
iam_setup.sh	Mock data, script or configuration

Detailed procedure

Step 1

Set PROJECT_ID and save the current project IAM policy as iam-policy-before.json.

Step 2

Create the user-managed service account app-runtime using the supplied script.

Step 3

Grant only roles/storage.objectViewer at project scope. Do not grant Owner or Editor.

Step 4

Save the new policy as iam-policy-after.json and compare the two files.

Step 5

Use IAM Policy Troubleshooter or a dry-run access test to check that the service account can read object metadata.

Step 6

Attempt a storage object delete with the service account. Record the expected permission-denied result.

Step 7

Inspect Cloud Audit Logs for the SetIamPolicy call and record principalEmail, methodName and resourceName.

Step 8

Remove any temporary human test binding. Keep the service account for Activities 4 and 7.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC

Evidence item	Acceptance criterion
	timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 3 Virtual Networks

Mapped criteria A3 · K4. ACE domain: Plan, deploy and manage networking.

Global VPC and regional subnet model

Architecture: global VPC, regional subnet, route table, firewall policy.

Runtime trace: select network → allocate CIDR → attach interface → route packet → enforce firewall.

```
gcloud compute networks create ace-vpc --subnet-mode=custom
gcloud compute networks subnets create app-sg --network=ace-vpc --region=asia-southeast1
--range=10.20.0.0/24
```

Acceptance evidence: Subnet range is unique, regional and visible in the intended global VPC.

Common failure modes: overlapping CIDR; auto-mode surprise; wrong region.

CIDR capacity and reserved addresses

Architecture: prefix length, host bits, primary range, secondary range.

Runtime trace: choose capacity → exclude overlap → reserve platform addresses → assign → monitor growth.

```
python3 -c 'import ipaddress; n=ipaddress.ip_network("10.20.0.0/24");
print(n.num_addresses)'
```

Acceptance evidence: Address plan records total range, usable design capacity and expansion path.

Common failure modes: undersized subnet; overlap with on-prem; secondary range omitted for GKE.

Route selection and next hops

Architecture: destination prefix, priority, next hop, instance interface.

Runtime trace: collect matching routes → longest prefix → priority → next hop → deliver.

```
gcloud compute routes list --filter='network:ace-vpc' --
format='table(name,destRange,priority,nextHopGateway,nextHopInstance)'
```

Acceptance evidence: The winning route has the most specific destination, then the applicable priority.

Common failure modes: missing custom route; asymmetric return; route exists but firewall denies.

Firewall policy evaluation

Architecture: direction, priority, target, source/destination, protocol and port.

Runtime trace: select applicable rules → lowest priority number → evaluate action → log decision → allow or deny.

```
gcloud compute firewall-rules create allow-web --network=ace-vpc --direction=INGRESS --
priority=1000 --action=ALLOW --rules=tcp:80 --source-ranges=10.20.0.0/16 --target-
tags=web
```

Acceptance evidence: Rule target and source range are narrower than 0.0.0.0/0 and firewall logging can prove the decision.

Common failure modes: wrong network tag; higher-priority deny; implied ingress deny.

Private Google Access and Cloud NAT

Architecture: internal-only VM, subnet setting, Google APIs, Cloud NAT.

Runtime trace: resolve destination → select private access or NAT → translate egress → return → log.

```
gcloud compute networks subnets update app-sg --region=asia-southeast1 --enable-private-ip-google-access
```

Acceptance evidence: VM without external IP can reach intended Google APIs; general internet egress uses controlled NAT.

Common failure modes: no default route; PGA disabled; NAT port exhaustion.

VPC Flow Logs as packet evidence

Architecture: 5-tuple, reporter, bytes, action context.

Runtime trace: sample flow → aggregate → write log → query → correlate.

```
gcloud logging read 'logName:vpc_flows AND jsonPayload.connection.src_ip="10.20.0.10"' --limit=20
```

Acceptance evidence: Flow record supports a connectivity hypothesis with source, destination, ports and bytes.

Common failure modes: sampling hides short flow; query wrong log bucket; firewall log confused with flow log.

Shared VPC ownership boundaries

Architecture: host project, service project, shared subnet, service agent.

Runtime trace: share network → delegate subnet → attach workload → enforce central policy → audit.

```
gcloud compute shared-vpc get-host-project $PROJECT_ID
```

Acceptance evidence: Network ownership and workload ownership are separately documented and permissioned.

Common failure modes: service project not associated; missing networkUser role; subnet in wrong host.

Connectivity Tests diagnostic graph

Architecture: source endpoint, configuration analysis, data plane probe, result path.

Runtime trace: define endpoints → analyze config → optional probe → identify hop → recommend fix.

```
gcloud network-management connectivity-tests list --format='table(name,source,destination)'
```

Acceptance evidence: Test result names the failing hop or policy instead of relying on ping alone.

Common failure modes: ICMP blocked but TCP works; stale test; source identity lacks visibility.

Activity 03 Build and Diagnose a Custom VPC

Duration 90 minutes. Mapped criteria A3, K4.

Build a custom VPC and regional subnets, apply narrow firewall rules and diagnose a failed connection.

Submit: network inventory, connectivity-test result and firewall evidence

Files supplied

File	Use
network_setup.sh	Mock data, script or configuration
network_plan.csv	Mock data, script or configuration

Detailed procedure

Step 1

Set the correct project and region asia-southeast1.

Step 2

Review network_plan.csv and confirm neither CIDR overlaps the default network or an assigned corporate range.

Step 3

Run network_setup.sh to create ace-vpc and app-sg.

Step 4

Create data-sg from the plan using a separate gcloud command.

Step 5

List the VPC, subnets, routes and firewall rules; save the tables as network-inventory.txt.

Step 6

Create two small test VMs without external IP addresses, one tagged web and one untagged.

Step 7

Serve a test page on the tagged VM and test TCP 80 from the second VM.

Step 8

Remove the web tag temporarily and repeat the test. Record the failure and identify the target mismatch.

Step 9

Restore the tag and create a Network Intelligence Center Connectivity Test between the endpoints.

Step 10

Enable or inspect firewall logging and preserve the allow/deny evidence.

Step 11

Stop the VMs after testing; keep the network for later activities.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 4 Virtual Machines

Mapped criteria A3 · K4. ACE domain: Deploy and operate Compute Engine.

Instance contract and dependencies

Architecture: machine type, boot disk image, NIC, service account.

Runtime trace: validate quota → allocate disk and NIC → boot → run guest agent → report status.

```
gcloud compute instances create web-1 --zone=asia-southeast1-b --machine-type=e2-micro
--subnet=app-sg --no-address --service-account=app-
runtime@$PROJECT_ID.iam.gserviceaccount.com --scopes=cloud-platform
```

Acceptance evidence: Instance RUNNING state is paired with the intended zone, network, disk and service account.

Common failure modes: quota exceeded; image unavailable; subnet region mismatch.

Machine families and right-sizing

Architecture: general purpose, compute optimized, memory optimized, accelerator.

Runtime trace: measure workload → select family → choose vCPU/RAM → benchmark → resize.

```
gcloud compute machine-types describe e2-standard-2 --zone=asia-southeast1-b
```

Acceptance evidence: Selection is justified by measured CPU, memory, latency and cost constraints.

Common failure modes: CPU platform assumption; oversizing; quota not available in zone.

Images, snapshots and persistent disks

Architecture: image family, boot disk, snapshot, new disk.

Runtime trace: write blocks → quiesce → snapshot → restore → verify filesystem.

```
gcloud compute disks snapshot web-1 --zone=asia-southeast1-b --snapshot-names=web-1-
prechange
```

Acceptance evidence: Snapshot has a source disk, creation time and restorable status before a risky change.

Common failure modes: crash-consistent only; wrong source disk; snapshot schedule absent.

Startup scripts and metadata execution

Architecture: instance metadata, guest agent, startup script, serial console.

Runtime trace: boot → read metadata → execute script → emit log → serve workload.

```
gcloud compute instances add-metadata web-1 --zone=asia-southeast1-b --metadata-from-
file=startup-script=startup.sh
```

Acceptance evidence: Serial console or journal output proves script execution and idempotent configuration.

Common failure modes: script not executable; package repository unavailable; non-idempotent rerun.

OS Login and SSH authorization

Architecture: IAM principal, OS Login role, POSIX account, SSH certificate.

Runtime trace: authenticate → authorize IAM → provision POSIX → mint key → connect.

```
gcloud compute ssh web-1 --zone=asia-southeast1-b --tunnel-through-iap
```

Acceptance evidence: Access succeeds without exposing TCP 22 publicly and is tied to an identifiable principal.

Common failure modes: OS Login disabled; IAP firewall missing; user lacks tunnel role.

Managed instance group control loop

Architecture: instance template, target size, health check, autoscaler.

Runtime trace: compare desired → create or recreate → health probe → scale → stabilize.

```
gcloud compute instance-groups managed list-instances web-mig --region=asia-southeast1
```

Acceptance evidence: All instances are current, healthy and derived from the intended template version.

Common failure modes: template drift; health check blocked; autoscaler cool-down too short.

Spot VM interruption-aware design

Architecture: Spot provision model, termination signal, checkpoint, durable store.

Runtime trace: schedule → process → receive termination → checkpoint → resume.

```
gcloud compute instances describe batch-spot --zone=asia-southeast1-b --  
format='value(scheduling.provisioningModel,scheduling.instanceTerminationAction) '
```

Acceptance evidence: Workload tolerates interruption and persists progress outside the instance.

Common failure modes: state on local disk; no retry queue; capacity unavailable.

VM incident evidence bundle

Architecture: instance status, serial logs, guest metrics, audit change.

Runtime trace: capture inventory → collect logs → correlate time → test hypothesis → escalate.

```
gcloud compute instances get-serial-port-output web-1 --zone=asia-southeast1-b --port=1  
> serial.log
```

Acceptance evidence: Escalation includes timestamps, recent changes, expected/observed behaviour and exact commands.

Common failure modes: reboot before evidence; timezone mismatch; missing reproduction.

Activity 04 Deploy and Recover a Compute Engine Web VM

Duration 90 minutes. Mapped criteria A3, K4.

Deploy a private web VM with a user-managed service account, test the startup script and recover from a bad change.

Submit: startup logs, a pre-change snapshot and recovery verification

Files supplied

File	Use
startup.sh	Mock data, script or configuration
verify_vm.sh	Mock data, script or configuration

Detailed procedure

Step 1

Review startup.sh and verify it is idempotent and contains no credentials.

Step 2

Create web-1 in asia-southeast1-b on app-sg without an external IP, attach app-runtime and add the web network tag.

Step 3

Supply startup.sh as instance metadata and wait until the VM is RUNNING.

Step 4

Run verify_vm.sh and inspect the serial output for package and service errors.

Step 5

Connect through IAP or an approved private path; do not create a public SSH rule.

Step 6

Run curl http://localhost and record the expected ACE activity healthy response.

Step 7

Create snapshot web-1-prechange before changing the web page or service configuration.

Step 8

Introduce the trainer-provided harmless configuration fault, restart Nginx and record systemctl plus journal output.

Step 9

Restore the known-good configuration or disk snapshot and repeat the curl test.

Step 10

Save the instance description, startup evidence, snapshot name and recovery result.

Step 11

Stop the VM when finished and verify Billing/Compute inventory.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 5 Storage and Databases

Mapped criteria A4 · K5. ACE domain: Plan, deploy and manage data solutions.

Cloud Storage request path

Architecture: bucket, object namespace, IAM, encryption and location.

Runtime trace: authenticate → authorize → select object → read or write → log.

```
gcloud storage buckets create gs://$BUCKET --location=asia-southeast1 --uniform-bucket-level-access
```

Acceptance evidence: Bucket location, uniform access and retention requirements are recorded before upload.

Common failure modes: globally unique name collision; wrong location; public access prevention conflict.

Storage class decision rules

Architecture: Standard, Nearline 30d, Coldline 90d, Archive 365d.

Runtime trace: measure access → choose class → store → retrieve → review lifecycle.

```
gcloud storage buckets describe gs://$BUCKET --format='value(storageClass,location,type)'
```

Acceptance evidence: Class matches expected access frequency and minimum-duration economics.

Common failure modes: early deletion charge; retrieval fee ignored; class used as durability control.

Lifecycle, versioning and retention

Architecture: lifecycle rule, object age, version state, retention lock.

Runtime trace: evaluate condition → apply action → retain evidence → expire → audit.

```
gcloud storage buckets update gs://$BUCKET --lifecycle-file=lifecycle.json
gcloud storage buckets update gs://$BUCKET --versioning
```

Acceptance evidence: Policy JSON is valid and noncurrent versions follow the intended retention window.

Common failure modes: delete rule too broad; locked policy irreversible; soft delete cost overlooked.

Cloud SQL private connectivity

Architecture: Cloud SQL instance, private service access, VPC, client.

Runtime trace: resolve private IP → route → authorize IAM or DB user → TLS session → query.

```
gcloud sql instances describe ace-db --format='value(ipAddresses,settings.availabilityType,settings.backupConfiguration.enabled)'
```

Acceptance evidence: Client reaches the private address and database authentication is distinct from network reachability.

Common failure modes: network not peered; connection limit; wrong database user.

Regional HA and failover

Architecture: primary zone, standby zone, synchronous replication, shared endpoint.

Runtime trace: commit → replicate → detect primary failure → promote standby → reconnect.

```
gcloud sql instances describe ace-db --  
format='value(settings.availabilityType,gceZone,secondaryGceZone) '
```

Acceptance evidence: REGIONAL availability and reconnect behaviour are proven by a controlled failover test.

Common failure modes: zonal instance; application no retry; long transaction during failover.

Database observability and query diagnosis

Architecture: Query Insights, CPU and memory metrics, connections, slow query.

Runtime trace: detect latency → rank query → inspect plan → tune → compare baseline.

```
gcloud monitoring time-series list  
--filter='metric.type="cloudsql.googleapis.com/database/cpu/utilization"' --limit=5
```

Acceptance evidence: Before/after latency, rows scanned and connection pressure support the chosen fix.

Common failure modes: scale hides bad query; pool leak; index write overhead.

Managed database selection

Architecture: Cloud SQL, Spanner, Bigtable, Firestore.

Runtime trace: model data → set consistency → set scale → choose interface → validate.

```
gcloud sql tiers list --filter='region:asia-southeast1' --  
format='table(tier,RAM,diskQuota) '
```

Acceptance evidence: Choice is based on transaction model, consistency, access pattern and operational boundary.

Common failure modes: product by brand familiarity; relational joins on Bigtable; global scale assumed free.

Backup, restore and data validation

Architecture: automated backup, point-in-time recovery, restore target, validation query.

Runtime trace: capture backup → select recovery point → restore → run checks → document RTO/RPO.

```
gcloud sql backups list --instance=ace-db  
gcloud sql operations list --instance=ace-db --limit=10
```

Acceptance evidence: Restored copy passes row-count, checksum and application connection tests.

Common failure modes: backup exists but restore untested; PITR log gap; restore over production.

Activity 05 Operate Cloud Storage and Diagnose Cloud SQL

Duration 105 minutes. Mapped criteria A4, K5.

Apply storage lifecycle controls, load mock customer data and diagnose a simulated Cloud SQL connection bottleneck.

Submit: lifecycle policy, query evidence and database incident report

Files supplied

File	Use
customers.csv	Mock data, script or configuration
lifecycle.json	Mock data, script or configuration
diagnose.sql	Mock data, script or configuration

Detailed procedure

Step 1

Set BUCKET to a globally unique lower-case name and create a regional bucket with uniform bucket-level access.

Step 2

Upload customers.csv and verify object size, generation and checksum.

Step 3

Apply lifecycle.json only after reviewing the 30-day condition and Nearline action.

Step 4

Enable versioning and upload a corrected copy of one record; list object generations.

Step 5

For the database path, use the trainer-provided Cloud SQL sandbox or mock outputs if live provisioning is not approved.

Step 6

Load the sample customer records into a training table and run row-count and region-grouping validation queries.

Step 7

Run diagnose.sql or interpret the supplied mock results to inspect connection states and slow statements.

Step 8

Reproduce a safe connection-pool saturation symptom with the trainer-provided harness.

Step 9

Apply one bounded remediation such as pool sizing, query index or instance connection limit, then repeat the same test.

Step 10

Document before/after connection count, mean execution time and error rate.

Step 11

Export the final query evidence and database incident report.

Step 12

Delete database resources if individually provisioned; retain only approved shared sandbox assets.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 6 Interconnecting Networks

Mapped criteria A4 · K5. ACE domain: Implement hybrid and multi-project connectivity.

Cloud VPN packet path

Architecture: HA VPN gateway, tunnel, Cloud Router, peer gateway.

Runtime trace: encrypt → send tunnel → decrypt → route by BGP → deliver.

```
gcloud compute vpn-tunnels list --format='table(name,region,status,peerIp,router)'
```

Acceptance evidence: Both HA interfaces and BGP sessions meet the intended redundancy design.

Common failure modes: single tunnel; shared secret mismatch; selector or route mismatch.

BGP dynamic route exchange

Architecture: Cloud Router, BGP peer, advertised prefix, learned route.

Runtime trace: establish session → advertise → learn → select route → withdraw on failure.

```
gcloud compute routers get-status ace-router --region=asia-southeast1 --format=json
```

Acceptance evidence: Peer status is UP and learned/advertised prefixes match the approved plan.

Common failure modes: ASN conflict; MD5/session mismatch; custom advertisement omits subnet.

Interconnect product decision

Architecture: Dedicated Interconnect, Partner Interconnect, Cross-Cloud Interconnect, VLAN attachment.

Runtime trace: set throughput/SLA → choose provider → provision circuit → attach VLAN → run BGP.

```
gcloud compute interconnects attachments list --  
format='table(name,region,state,type,router)'
```

Acceptance evidence: Selection reflects throughput, location, provider, encryption and SLA requirements.

Common failure modes: no redundant metro; VLAN attachment inactive; BGP not established.

VPC Network Peering semantics

Architecture: peer VPC A, peer VPC B, subnet routes, non-transitive boundary.

Runtime trace: create both sides → exchange eligible routes → enforce firewall → send directly → observe.

```
gcloud compute networks peerings list --network=ace-vpc
```

Acceptance evidence: Both sides are ACTIVE and no design depends on transitive routing.

Common failure modes: overlapping ranges; one side missing; firewall not automatically shared.

Shared VPC versus peering

Architecture: central host project, service projects, separate admin roles, shared subnets.

Runtime trace: choose ownership model → delegate → attach workload → apply policy → audit.

```
gcloud compute shared-vpc associated-projects list $HOST_PROJECT
```

Acceptance evidence: Chosen model matches whether central administrators must own the network dataplane.

Common failure modes: using peering for centralized subnet ownership; host service agent missing; project association drift.

Private Service Connect service boundary

Architecture: service attachment, consumer endpoint, internal IP, producer service.

Runtime trace: publish → approve consumer → create endpoint → connect privately → monitor.

```
gcloud compute forwarding-rules list --filter='target:serviceAttachments' --  
format='table(name,IPAddress,network,region) '
```

Acceptance evidence: Consumer uses a private endpoint without importing the producer's routes.

Common failure modes: connection not accepted; wrong region; DNS still resolves public endpoint.

Hybrid DNS resolution

Architecture: Cloud DNS private zone, inbound forwarding, outbound forwarding, on-prem resolver.

Runtime trace: query → match zone → forward if needed → return answer → cache.

```
gcloud dns managed-zones list --format='table(name,dnsName,visibility) '
```

Acceptance evidence: Resolution path and authoritative zone are testable from both cloud and on-prem sources.

Common failure modes: overlapping zone suffix; forwarder unreachable; stale cache.

Hybrid connectivity fault isolation

Architecture: interface, BGP session, route, firewall, application port.

Runtime trace: check link → check session → check route → check policy → test application.

```
gcloud compute routers get-status ace-router --region=asia-southeast1  
gcloud network-management connectivity-tests list
```

Acceptance evidence: Evidence isolates control-plane, routing, policy or application failure before escalation.

Common failure modes: testing only ping; asymmetric route; MTU black hole.

Activity 06 Model Hybrid Connectivity and BGP Failure

Duration 90 minutes. Mapped criteria A4, K5.

Evaluate VPN versus Interconnect, inspect mock BGP routes and isolate a hybrid routing failure.

Submit: connectivity decision, route analysis and corrected BGP configuration

Files supplied

File	Use
bgp_routes.csv	Mock data, script or configuration
router_config.yaml	Mock data, script or configuration

Detailed procedure

Step 1

Read the scenario: an on-premises 172.16.0.0/16 network must reach ace-vpc with predictable failover.

Step 2

Complete a decision table comparing HA VPN, Partner Interconnect and Dedicated Interconnect for throughput, SLA, lead time and cost.

Step 3

Inspect router_config.yaml and confirm local ASN 64514 differs from peer ASN 65010.

Step 4

Open bgp_routes.csv and identify the most specific route for 10.20.10.0/24.

Step 5

Simulate a missing 10.20.10.0/24 advertisement and explain the traffic consequence.

Step 6

Use a trainer sandbox or mock gcloud output to inspect VPN tunnel status and Cloud Router BGP peer status.

Step 7

Trace one packet from 172.16.0.10 to 10.20.10.20 through peer gateway, tunnel, Cloud Router, VPC route and firewall.

Step 8

Correct the advertisement or route priority in a copy of the YAML/CSV.

Step 9

Run a configuration comparison and save the before/after result.

Step 10

Prepare a short hybrid-network incident note including the failed hop, evidence and escalation owner.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 7 Managed Services

Mapped criteria A3 · K4. ACE domain: Deploy serverless and data services.

Cloud Run revision and traffic model

Architecture: service, revision, container instance, traffic split.

Runtime trace: deploy image → create immutable revision → start instances → route traffic → observe.

```
gcloud run deploy ace-api --source=. --region=asia-southeast1 --no-allow-unauthenticated
```

Acceptance evidence: Revision name, image digest, service identity and ingress are captured after deployment.

Common failure modes: container not listening on PORT; startup timeout; unauthorized invocation.

Cloud Run autoscaling controls

Architecture: CPU, concurrency, min instances, max instances.

Runtime trace: receive load → measure drivers → calculate recommendation → bound count → scale.

```
gcloud run services update ace-api --region=asia-southeast1 --concurrency=40 --min=0 --max=10
```

Acceptance evidence: Instance count and latency respond to load without overwhelming the database connection budget.

Common failure modes: single-thread vCPU hotspot; max too high for DB; cold-start latency.

Pub/Sub delivery contract

Architecture: topic, subscription, message, ack deadline.

Runtime trace: publish → store → deliver → ack or retry → dead-letter.

```
gcloud pubsub topics create orders
gcloud pubsub subscriptions create orders-worker --topic=orders --ack-deadline=30
```

Acceptance evidence: Subscriber handles duplicate delivery idempotently and backlog metrics remain within target.

Common failure modes: no ack; poison message loop; publisher permission missing.

Eventarc event routing

Architecture: event provider, Eventarc trigger, transport topic, Cloud Run target.

Runtime trace: emit event → match filters → wrap CloudEvent → invoke target → record delivery.

```
gcloud eventarc triggers list --location=asia-southeast1 --format='table(name,eventFilters,destination)'
```

Acceptance evidence: Trigger filters match the intended event type and target region.

Common failure modes: region mismatch; service agent role missing; event filter typo.

BigQuery load and query job model

Architecture: dataset, table schema, load job, query job.

Runtime trace: stage data → submit job → scan columns/partitions → write result → inspect bytes.

```
bq load --source_format=CSV --autodetect ops.events gs://$BUCKET/events.csv
bq query --use_legacy_sql=false 'SELECT severity,COUNT(*) n FROM `ops.events` GROUP BY severity'
```

Acceptance evidence: Job status is DONE with no error and bytes processed are reviewed before scale-up.

Common failure modes: schema autodetect drift; unbounded scan; dataset location mismatch.

Dataflow pipeline boundaries

Architecture: source, Apache Beam transforms, runner workers, sink.

Runtime trace: read → window/key → transform → shuffle → write.

```
gcloud dataflow jobs list --region=asia-southeast1 --status=active
```

Acceptance evidence: Job graph, worker logs, watermark and failed records show pipeline health.

Common failure modes: hot key; serialization failure; sink quota.

Dataproc cluster and job separation

Architecture: cluster, driver, executors, Cloud Storage.

Runtime trace: provision → submit job → schedule stages → persist output → delete cluster.

```
gcloud dataproc jobs list --region=asia-southeast1 --  
format='table(reference.jobId,status.state,placement.clusterName) '
```

Acceptance evidence: Ephemeral compute is separated from durable input/output and job state is recorded.

Common failure modes: cluster left running; initialization action fails; shuffle disk pressure.

Serverless incident trace

Architecture: request log, revision, trace ID, downstream call.

Runtime trace: receive request → emit structured log → propagate trace → call dependency → correlate.

```
gcloud logging read 'resource.type="cloud_run_revision" AND  
resource.labels.service_name="ace-api"' --limit=30 --format=json
```

Acceptance evidence: One trace identifier links entry, application error and downstream latency.

Common failure modes: unstructured logs; trace header dropped; wrong revision receives traffic.

Activity 07 Build a Pub Sub to Cloud Run Event Pipeline

Duration 105 minutes. Mapped criteria A3, A4, K4.

Deploy a small event receiver, publish mock orders and diagnose retry behaviour without losing messages.

Submit: Cloud Run revision, Pub/Sub subscription and processed event log

Files supplied

File	Use
app.py	Mock data, script or configuration
requirements.txt	Mock data, script or configuration
orders.jsonl	Mock data, script or configuration

Detailed procedure

Step 1

Review app.py, requirements.txt and orders.jsonl. Confirm the receiver logs message ID and payload but no secret.

Step 2

Create the orders topic and a subscription, then publish one test message manually.

Step 3

Build and deploy ace-api to Cloud Run in asia-southeast1 with authenticated ingress.

Step 4

Create a Pub/Sub push subscription or Eventarc trigger using the Cloud Run service identity and documented invoker permissions.

Step 5

Publish both mock orders and query Cloud Run revision logs for the two message IDs.

Step 6

Send the same message ID twice in the mock harness and record the duplicate-delivery risk.

Step 7

Add or describe an idempotency control keyed on order_id/message ID.

Step 8

Introduce a harmless 500 response for one test event and observe retry/backlog behaviour.

Step 9

Restore the 204 response and verify the backlog returns to zero.

Step 10

Save the revision name, trigger/subscription configuration and processed event evidence.

Step 11

Delete the service, topic and subscription unless the trainer asks you to retain them.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 8 Kubernetes

Mapped criteria A3 · K4. ACE domain: Deploy and operate GKE.

GKE control plane and workload boundary

Architecture: managed control plane, node or Autopilot compute, Pod, Service.

Runtime trace: submit manifest → admission → schedule → start container → reconcile.

```
gcloud container clusters create-auto ace-gke --region=asia-southeast1
gcloud container clusters get-credentials ace-gke --region=asia-southeast1
```

Acceptance evidence: Cluster endpoint is reachable and credentials bind kubectl to the intended context.

Common failure modes: wrong context; API disabled; regional quota.

Kubernetes declarative reconciliation

Architecture: desired manifest, API server, controller, observed state.

Runtime trace: apply → persist desired state → controller diff → act → converge.

```
kubectl apply -f deployment.yaml
kubectl get deploy,pods,svc -o wide
```

Acceptance evidence: Deployment AVAILABLE replicas equal desired replicas and Pods are Ready.

Common failure modes: selector mismatch; image pull error; readiness probe fails.

Resource requests, limits and scheduling

Architecture: Pod request, scheduler, node capacity, container limit.

Runtime trace: sum requests → filter nodes → score → bind Pod → enforce cgroup.

```
kubectl describe pod -l app=ace-web
kubectl top pods
```

Acceptance evidence: Requests reflect observed workload and explain both scheduling and autoscaling decisions.

Common failure modes: Pending due to insufficient CPU; OOMKilled; no metrics.

Service discovery and traffic

Architecture: Service selector, EndpointSlice, cluster IP, Pod targetPort.

Runtime trace: resolve service DNS → select endpoint → proxy packet → reach Pod → respond.

```
kubectl get svc,endpointlices -o wide
kubectl describe svc ace-web
```

Acceptance evidence: Selector labels produce ready endpoints and port/targetPort match the container.

Common failure modes: zero endpoints; wrong targetPort; NetworkPolicy blocks flow.

Artifact Registry image pull

Architecture: repository, image digest, node identity, Pod spec.

Runtime trace: push image → authorize pull → resolve tag → download layers → start.

```
gcloud artifacts docker images list asia-southeast1-docker.pkg.dev/$PROJECT_ID/ace/apps
--include-tags
```

Acceptance evidence: Running Pod imageID records the immutable digest expected from the build.

Common failure modes: repository region typo; node lacks reader role; mutable tag drift.

Horizontal Pod Autoscaler control loop

Architecture: metrics server, target utilization, replica recommendation, Deployment.

Runtime trace: sample metrics → calculate ratio → stabilize → scale replicas → reschedule.

```
kubectl autoscale deployment ace-web --cpu-percent=60 --min=2 --max=10
kubectl get hpa -w
```

Acceptance evidence: HPA shows current/target metric and increases replicas during the load window.

Common failure modes: CPU requests missing; metrics unavailable; load too short.

Cluster autoscaling and Autopilot

Architecture: unschedulable Pod, requested resources, Autopilot provisioner, new capacity.

Runtime trace: Pod pending → detect need → provision node → schedule → scale down.

```
kubectl get events --sort-by=.lastTimestamp | tail -30
```

Acceptance evidence: Events distinguish Pod HPA from underlying capacity provisioning.

Common failure modes: unsupported workload constraint; quota exhausted; first workload provisioning delay.

GKE troubleshooting evidence

Architecture: Pod status, events, container logs, cluster metrics.

Runtime trace: identify symptom → describe resource → read current/previous log → test service → escalate.

```
kubectl describe pod $POD
kubectl logs $POD --previous
kubectl get events --sort-by=.lastTimestamp
```

Acceptance evidence: Incident bundle preserves YAML, events, logs, metrics and the exact failed test.

Common failure modes: Pod recreated before capture; namespace omitted; only application log collected.

Activity 08 Deploy and Test a GKE Autopilot Workload

Duration 120 minutes. Mapped criteria A3, A4, A5.

Deploy a containerized web workload with probes and resource requests, add HPA and diagnose why replicas do not scale.

Submit: Kubernetes manifests, load-test CSV and HPA evidence

Files supplied

File	Use
deployment.yaml	Mock data, script or configuration
hpa.yaml	Mock data, script or configuration

Detailed procedure

Step 1

Set the project and region, enable the GKE API and create ace-gke in Autopilot mode.

Step 2

Fetch credentials and run `kubectl config current-context`; confirm the project, cluster and region.

Step 3

Review `deployment.yaml` for image, labels, ports, resource requests/limits and readiness probe.

Step 4

Apply `deployment.yaml` and wait for both replicas to become Ready.

Step 5

Apply `hpa.yaml`. Run `kubectl get hpa` and confirm the TARGETS column has metrics rather than unknown.

Step 6

Generate controlled HTTP load from an approved load generator for at least five minutes.

Step 7

In a second terminal, record HPA current/target CPU, desired replicas and Pod count every 30 seconds.

Step 8

If replicas remain unchanged, check CPU requests, Metrics API output, workload intensity, `maxReplicas` and recent events.

Step 9

Correct exactly one identified cause and repeat the same load profile.

Step 10

Save `kubectl describe hpa`, events, Pod status and the load-test CSV.

Step 11

Scale the workload down or delete the cluster after evidence is captured to avoid ongoing cost.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 9 Resource Management and Monitoring

Mapped criteria A5 · K2. ACE domain: Ensure successful operation.

Asset inventory and labels

Architecture: Cloud Asset Inventory, resource name, labels, search query.

Runtime trace: discover → normalize metadata → filter → export → review.

```
gcloud asset search-all-resources --scope=projects/$PROJECT_NUMBER --  
query='labels.env=training' --format='table(assetType,name,location)'
```

Acceptance evidence: Inventory finds all expected resources and exposes unlabelled or wrong-location assets.

Common failure modes: label key inconsistency; scope too narrow; deleted asset timing.

Quota versus system limit

Architecture: quota metric, regional dimension, consumer override, hard limit.

Runtime trace: measure usage → compare quota → forecast → request increase → verify.

```
gcloud compute regions describe asia-southeast1 --  
format='table(quotas.metric,quotas.usage,quotas.limit)'
```

Acceptance evidence: Capacity plan identifies the quota and region before deployment pressure.

Common failure modes: quota confused with capacity; request too late; per-project versus per-region.

Metrics, logs and traces

Architecture: time series, log entry, trace span, resource labels.

Runtime trace: instrument → ingest → index → correlate → interpret.

```
gcloud monitoring metrics list --filter='metric.type:compute.googleapis.com' --limit=10
```

Acceptance evidence: Signal includes resource type, metric/log name, timestamp and useful labels.

Common failure modes: metric has wrong alignment; logs lack context; trace sampling gap.

Cloud Logging router

Architecture: source resource, Log Router, sink filter, destination.

Runtime trace: receive log → match exclusion/sink → route → store → query.

```
gcloud logging sinks list --format='table(name,destination,filter)'
```

Acceptance evidence: Sink destination and filter are explicit; required audit logs remain retained.

Common failure modes: recursive routing; writer identity lacks destination role; exclusion drops evidence.

Log-based metric contract

Architecture: log filter, counter metric, time series, alert policy.

Runtime trace: match entry → increment metric → align window → evaluate → notify.

```
gcloud logging metrics create app-errors --description='ACE app errors' --log-  
filter='severity>=ERROR AND resource.type="cloud_run_revision"'
```

Acceptance evidence: Test error increments the metric and can be tied to a known trace or revision.

Common failure modes: filter never matches; high-cardinality label; ingestion delay misread.

Alert policy evaluation

Architecture: metric filter, alignment window, threshold, notification channel.

Runtime trace: ingest points → align → compare → open incident → notify and close.

```
gcloud alpha monitoring policies list --format='table(displayName,enabled,conditions)'
```

Acceptance evidence: Synthetic breach opens one incident and recovery closes it without alert flapping.

Common failure modes: window too short; missing data behaviour; channel unverified.

Ops Agent telemetry path

Architecture: application/system, Ops Agent receiver, processor, Logging/Monitoring.

Runtime trace: collect → parse → enrich → export → query.

```
sudo systemctl status google-cloud-ops-agent
sudo /opt/stackdriver/stack-config/config-checker --config-path=/etc/google-cloud-ops-agent/config.yaml
```

Acceptance evidence: Agent service is healthy and expected log/metric names appear with resource labels.

Common failure modes: invalid YAML; agent service account lacks role; duplicate ingestion.

Evidence-led escalation

Architecture: impact, timeline, metrics and logs, owner and next action.

Runtime trace: stabilize → capture → compare expected → attempt safe fix → escalate.

```
gcloud logging read 'timestamp>=\"2026-09-12T00:00:00Z\" AND severity>=ERROR' --limit=100 --format=json > incident-logs.json
```

Acceptance evidence: Escalation pack is reproducible, secret-safe and states what remains unknown.

Common failure modes: screenshots without timestamps; credentials in logs; no rollback record.

Activity 09 Create Monitoring Evidence and an Incident Pack

Duration 90 minutes. Mapped criteria A5, K2.

Create a log-based error metric and alert policy, trigger a synthetic fault and assemble escalation evidence.

Submit: alert-policy.json, incident timeline and filtered log bundle

Files supplied

File	Use
alert-policy.json	Mock data, script or configuration
incident_timeline.csv	Mock data, script or configuration

Detailed procedure

Step 1

Select one Activity 7 or 8 workload and define the expected signal, threshold, evaluation window and notification owner.

Step 2

Create a log-based counter metric whose filter matches a unique synthetic error marker.

Step 3

Create or review alert-policy.json; replace placeholders with the actual project resource name and notification channel.

Step 4

Trigger one error and verify it appears in Cloud Logging with the correct resource labels.

Step 5

Trigger enough errors to breach the threshold for the specified duration.

Step 6

Confirm an incident opens and the notification reaches the test channel.

Step 7

Collect the filtered log entries, metric chart and policy configuration without credentials.

Step 8

Stop the errors and confirm the incident closes after the recovery window.

Step 9

Complete incident_timeline.csv with UTC timestamps and evidence filenames.

Step 10

Write the escalation summary: impact, expected/observed behaviour, safe actions, remaining unknowns and owner.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Topic 10 Load Balancing and Autoscaling

Mapped criteria A6 · K2. ACE domain: Operate scalable and highly available services.

Application versus Network Load Balancers

Architecture: Layer 7 application, Layer 4 proxy, Layer 4 passthrough, internal/external.

Runtime trace: classify protocol → choose scope → choose proxy boundary → select backend → validate.

```
gcloud compute forwarding-rules list --  
format='table(name,loadBalancingScheme,IPProtocol,IPAddress,backendService)'
```

Acceptance evidence: Choice states OSI layer, client reachability, scope and source-IP requirement.

Common failure modes: HTTP on passthrough without reason; global/regional mismatch; wrong scheme.

Application Load Balancer request path

Architecture: forwarding rule, target proxy, URL map, backend service, health check.

Runtime trace: accept anycast IP → terminate TLS → match host/path → select healthy backend → proxy request.

```
gcloud compute url-maps describe ace-map --global  
gcloud compute backend-services get-health ace-backend --global
```

Acceptance evidence: URL-map route and backend health explain where a request is sent.

Common failure modes: certificate not active; URL rule mismatch; health check firewall blocked.

Health checks versus application readiness

Architecture: probe source, protocol/port, request path, backend response.

Runtime trace: probe → evaluate threshold → mark healthy → admit traffic → continue probing.

```
gcloud compute health-checks describe ace-hc --global
```

Acceptance evidence: Probe path tests a dependency-light readiness endpoint and healthy thresholds match recovery needs.

Common failure modes: probe hits redirect; app listens different port; probe requires authentication.

Managed instance group autoscaler

Architecture: metric, target, cooldown, min/max replicas.

Runtime trace: sample → recommend → bound → resize MIG → stabilize.

```
gcloud compute instance-groups managed set-autoscaling web-mig --region=asia-southeast1  
--min-num-replicas=2 --max-num-replicas=10 --target-cpu-utilization=0.60 --cool-down-  
period=90
```

Acceptance evidence: Observed group size rises under sustained load and returns after the stabilization period.

Common failure modes: load below target; cooldown masks change; max quota reached.

Capacity math and backend saturation

Architecture: requests per second, service time, concurrency, instance capacity.

Runtime trace: measure service time → derive concurrency → apply headroom → set max → load test.

```
python3 -c 'rps=240; seconds=.25; headroom=.7; print(round((rps*seconds)/headroom,1)) '
```

Acceptance evidence: Capacity estimate is compared with p95 latency and error rate from a controlled test.

Common failure modes: average hides p95; downstream bottleneck; no warm-up.

Traffic distribution and session affinity

Architecture: client tuple, hash/affinity, healthy backend set, connection tracking.

Runtime trace: build key → select backend → track flow → reuse → rebalance on failure.

```
gcloud compute backend-services describe ace-backend --global --  
format='value(sessionAffinity,localityLbPolicy,timeoutSec) '
```

Acceptance evidence: Affinity is enabled only when application state or protocol requires it.

Common failure modes: sticky session hides imbalance; source IP changes through proxy; state not externalized.

Autoscaling load-test evidence

Architecture: load generator, Cloud Monitoring, MIG size, backend latency.

Runtime trace: baseline → ramp → hold → observe scale → recover.

```
bash load_test.sh --url=$LB_URL --rps=240 --duration=300 > load-results.csv
```

Acceptance evidence: Test records target RPS, actual RPS, p95 latency, error rate and instance count over time.

Common failure modes: client becomes bottleneck; DNS cached old IP; test too short.

Escalation when scaling fails

Architecture: autoscaler status, quota, health, audit log.

Runtime trace: capture config → collect status → rule out quota → preserve load trace → escalate.

```
gcloud compute instance-groups managed describe web-mig --region=asia-southeast1 --  
format=json > mig.json  
gcloud compute operations list --filter='status!=DONE'
```

Acceptance evidence: Escalation identifies expected trigger, observed signal, blocking condition and safe attempts already made.

Common failure modes: manual resize destroys evidence; wrong region; quota error omitted.

Activity 10 Load Test a Managed Instance Group and Escalate

Duration 120 minutes. Mapped criteria A5, A6, K2.

Apply health checking and CPU autoscaling to a managed instance group, run a controlled load test and escalate a scaling failure.

Submit: load-results.csv, MIG configuration and escalation report

Files supplied

File	Use
load_test.sh	Mock data, script or configuration
load-results.csv	Mock data, script or configuration
escalation-template.md	Mock data, script or configuration

Detailed procedure

Step 1

Use an approved managed instance group sandbox or create a small regional MIG from the Activity 4 template.

Step 2

Attach a health check whose path returns 200 without depending on a slow external service.

Step 3

Configure minimum 2, maximum 10, target CPU 60% and cooldown 90 seconds.

Step 4

Record the healthy baseline: group size, backend health, p95 latency and error rate.

Step 5

Run load_test.sh against the load-balancer URL and monitor group size plus CPU in parallel.

Step 6

Hold the load long enough to exceed the target beyond the cooldown. Record actual scale-out time.

Step 7

If scaling fails, inspect autoscaler status, quota, managed operations, health check and instance template.

Step 8

Apply one safe correction and repeat the identical load profile.

Step 9

Allow the group to recover and record scale-in behaviour.

Step 10

Complete load-results.csv with observed values and attach MIG/autoscaler JSON.

Step 11

Use escalation-template.md if the target still fails; identify the next owner and preserve evidence.

Step 12

Delete the load balancer and MIG or set the trainer-approved retained state. Verify final cost inventory.

Verification and evidence

Evidence item	Acceptance criterion
Scope	Correct account, project and region/zone
Configuration	Current resource or policy output captured
Runtime	Expected and observed output with UTC timestamp
Failure test	One controlled negative result explained
Cleanup	Final resource inventory and cost check

Troubleshooting and escalation

Check identity and project first, then API enablement and IAM, resource state, networking, quotas and service-specific logs. Preserve the original failure output before changing configuration. Escalate with impact, timeline, exact command, expected result, observed result, evidence files, safe actions attempted and the requested next action.

Assessment Preparation

The Written Assessment contains five open-ended short-answer questions mapped to K1–K5. The Practical Performance contains three coherent tasks mapped to A1–A6. Rehearse the evidence pattern: scope, configure, test, compare, diagnose, remediate, verify and escalate.

References

Associate Cloud Engineer certification exam guide:

<https://cloud.google.com/learn/certification/guides/cloud-engineer>

Get started with Google Cloud: <https://docs.cloud.google.com/docs/get-started>

Google Cloud documentation: <https://docs.cloud.google.com/docs>

Google Cloud resource hierarchy: <https://docs.cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy>

IAM overview: <https://docs.cloud.google.com/iam/docs/overview>

VPC networks: <https://docs.cloud.google.com/vpc/docs/vpc>

Cloud Storage classes: <https://docs.cloud.google.com/storage/docs/storage-classes>

GKE Autopilot overview: <https://docs.cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview>

Cloud Run autoscaling: <https://docs.cloud.google.com/run/docs/about-instance-autoscaling>

Cloud Load Balancing overview: <https://docs.cloud.google.com/load-balancing/docs/load-balancing-overview>

Cloud Logging routing: <https://docs.cloud.google.com/logging/docs/routing/overview>

Network Connectivity product selection: <https://docs.cloud.google.com/network-connectivity/docs/how-to/choose-product>

Google Cloud Skills learning path: <https://www.skills.google/paths/8>

Google Cloud Codelabs: <https://codelabs.developers.google.com/cloud>

Google Cloud quickstarts: <https://cloud.google.com/docs/tutorials?category=compute,highperformancecomputing,migrations,networking,storage&doctype=quickstart>

GeeksforGeeks Google Cloud Platform tutorial: <https://www.geeksforgeeks.org/devops/google-cloud-platform-tutorial/>

Edureka Google Cloud Platform tutorial: <https://www.edureka.co/blog/google-cloud-platform-tutorial/>

freeCodeCamp Google Cloud Platform from zero to hero: <https://www.freecodecamp.org/news/google-cloud-platform-from-zero-to-hero/>